

DERİN KAZI

Geliştirme Yol Haritası — Detaylı Özellik ve Teknik Uygulama Raporu

Kapsam: Öncelikli Altyapı · Oynanış Derinliği · Multiplayer & Sosyal · Monetizasyon | Tarih: 23 Ağustos 2026

Bu rapor, "Derin Kazi" oyununa eklenebilecek özellikleri dört kategoride, her biri için hem oyun içi özellik açıklaması hem de Flutter/Dart teknik uygulama detaylarıyla birlikte sunar. Cilalama (ses, animasyon, tutorial) kategorisi bu raporun kapsamı dışında tutulmuştur.

DÜŞÜK	1-2 gün · tek dosya/ufak değişiklik	ORTA	3-7 gün · yeni model/servis gerekir
YÜKSEK	1-3 hafta · mimari değişiklik / backend gerekir		

İçindekiler

1	Öncelikli — Mevcut Altyapıyı Tamamlama	5 özellik
2	Oynanış Derinliği	6 özellik
3	Multiplayer ve Sosyal Özellikler	4 özellik
4	Monetizasyon ve Yaşam Döngüsü	3 özellik

Bölüm 1 — Öncelikli: Mevcut Altyapıyı Tamamlama

Kodda zaten kısmen hazır olan ama bağlanmamış özellikler. En yüksek etki / en düşük efor oranı.

#1 Kalıcı Kayıt Sistemi (Save / Load)

ORTA

Özellik: Oyuncunun altın, elmas, rütbe, envanter, yükseltme seviyeleri ve en yüksek derinlik bilgisi uygulama kapansa dahi korunur. Oyuncu kaldığı yerden devam eder.

Gerekli Paketler: `shared_preferences: ^2.3.0`

Etkilenen/Yeni Dosyalar:

- `lib/core/persistence/save_service.dart` (yeni)
- `lib/features/mining/application/game_notifier.dart` (düzenleme)
- `lib/main.dart` (düzenleme — açılışta yükleme)

Teknik Uygulama:

- `PlayerStateModel` ve `UpgradeModel`'de zaten `toJson()/fromJson()` var — bu doğrudan kullanılabilir.
- `GameNotifier` içine `_saveState()` metodu eklenip her state değişikliğinde (debounce ile) çağrılır.
- Uygulama açılışında `main.dart`'ta `ProviderScope` öncesi kayıtlı veri okunup `GameNotifier`'a enjekte edilir.
- Basit key-value ihtiyacı için `Hive` yerine `SharedPreferences` yeterli; ileride bulut senkronu istenirse `Hive+Firebase`'e geçilebilir.

```
class SaveService {
  static Future save(PlayerStateModel p) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('player', jsonEncode(p.toJson()));
  }
}
```

#2 Eksik Tile Tiplerini Grid Üretimine Bağlama

DÜŞÜK

Özellik: chest, emeraldOre, bossCore, tnt, specialItem tipleri zaten TileType enum'ında tanımlı ve tile_gridPainter.dart'ta çizim kodları hazır. Sadece üretim olasılıkları eklenerek bu içerikler oyunda gerçekten karşımıza çıkar.

Etkilenen/Yeni Dosyalar:

- lib/features/mining/domain/services/grid_generator.dart (düzenleme)

Teknik Uygulama:

- grid_generator.dart içindeki lootRoll switch/if zincirine yeni olasılık aralıkları eklenir (ör. 0.65-0.70 aralığı chest, 0.70-0.74 emeraldOre gibi).
- chest için rewardGold/rewardGems çok daha yüksek değerlerle set edilir.
- bossCore, stage % 10 == 0 gibi belirli derinliklerde grid'e tek adet, sabit HP'li olarak yerleştirilir.
- tnt kutusu kırıldığında zaten mevcut 3x3 patlama mantığı (Bulgu #3 düzeltilmiş hali ile) devreye girer.

```
} else if (lootRoll < 0.70) {  
  // Hazine Sandığı  
  rewardGold = 150 + stage * 25;  
  rewardGems = 2 + random.nextInt(3);  
  tileType = TileType.chest;  
}
```

#3 Maden Ödül Dağıtımını Aktive Etme

DÜŞÜK

Özellik: rewardCopper, rewardIron, rewardEmerald, rewardFossil, rewardDynamite alanları zaten model ve envanter arayüzünde var; sadece grid_generator.dart'ta değer atanması eksik.

Etkilenen/Yeni Dosyalar:

- lib/features/mining/domain/services/grid_generator.dart (düzenleme)

Teknik Uygulama:

- Mevcut lootRoll dağılımına yeni dallar eklenir: örn. rock tipi kutularda %20 ihtimalle rewardCopper = 1-3, %10 ihtimalle rewardIron = 1-2.
- emeraldOre tipi kutularda rewardEmerald = 1, nadir fossil ödülü tüm kutularda %2 ihtimalle.
- dynamite ödülü chest/bossCore gibi nadir tile'larda 1 adet olarak verilir.
- Bu değişiklik game_notifier.dart'ta ek kod gerektirmez — mevcut digTargetTile() mantığı zaten bu alanları PlayerStateModel'e ekliyor.

#4 Victory Podium Ekranını Devreye Alma

ORTA

Özellik: victory_podium_dialog.dart ve PlayerContributionModel hazır ama hiç çağrılmıyor. Takım modunda (coopCampaign) sefer/hedef derinliğe ulaşıldığında oyuncu katkı sıralamasını gösteren podyum ekranı açılır.

Etkilenen/Yeni Dosyalar:

- lib/features/mining/application/game_notifier.dart (düzenleme — victory kontrolü)
- lib/features/mining/presentation/screens/mining_screen.dart (düzenleme — dialog tetikleme)

Teknik Uygulama:

- GameNotifier'a _checkCoopVictory() metodu eklenir; RoomModel.targetDepth'e ulaşıldığında tetiklenir.
- otherPlayers listesindeki damageDealt/tilesCleared/goldEarned alanları kullanılarak List oluşturulur ve goldEarned'e göre sıralanır.
- En yüksek katkıyı yapan oyuncuya isMvp: true atanır.
- mining_screen.dart içindeki ref.listen bloğuna yeni bir victory state kontrolü eklenir (mevcut battlePhase.isFinished dinleme mantığına benzer şekilde).

#5 Solo / Multiplayer Battle Royale Ayrımı

DÜŞÜK

Özellik: Şu an GameNotifier constructor'ı her modda battle royale sayacını zorla başlatıyor. Solo modda bu sayaç tamamen devre dışı bırakılır, sadece gerçek çok oyunculu/battle royale odalarında çalışır.

Etkilenen/Yeni Dosyalar:

- lib/features/mining/application/game_notifier.dart (düzenleme)

Teknik Uygulama:

- GameNotifier constructor'ına isBattleRoyaleMode: bool parametresi eklenir, varsayılan false.
- startBattleRoyaleMatch() çağrısı yalnızca bu flag true iken constructor'dan tetiklenir.
- MainMenuScreen'deki 'SOLO KAZI' butonu GameNotifier'ı isBattleRoyaleMode: false ile başlatır; LobbyScreen'deki 'KAZIYA BAŞLA' (battle royale odaları için) true ile başlatır.

```
GameNotifier({this.isBattleRoyaleMode = false}) : super(...) {  
  _startEnergyTimer();  
  if (isBattleRoyaleMode) startBattleRoyaleMatch();  
}
```

Bölüm 2 — Oynanış Derinliği

Oyuncunun uzun vadeli ilgisini artıran, tekrar oynanabilirliği yükselten mekanikler.

#6 Boss Savaşları

ORTA

Özellik: Belirli derinliklerde (ör. her 10 katmanda bir) 500 HP'lik bossCore tile'ı ile özel bir karşılaşma. Boss kırıldığında büyük miktarda altın, elmas ve garanti bir alet ödülü verir.

Etkilenen/Yeni Dosyalar:

- lib/features/mining/domain/services/grid_generator.dart (düzenleme)
- lib/features/mining/application/game_notifier.dart (düzenleme)
- lib/features/mining/presentation/widgets/boss_health_bar.dart (yeni, opsiyonel)

Teknik Uygulama:

- GridGenerator.generateStage()'e isBossStage parametresi zaten var ama kullanılmıyor — bu parametre true olduğunda grid'in merkezine tek bir bossCore tile'ı yerleştirilir.
- digTargetTile() içine boss'a özel bir dal eklenir: kırıldığında ekstra bir 'BOSS YENİLDİ!' mesajı ve garanti diamondPick ödülü verir.
- İsteğe bağlı: boss'un HP barının ekranın üstünde ayrı bir widget ile gösterilmesi (BossHealthBar) oyuncuya ilerlemeyi hissettirir.

#7 Günlük Görevler (Daily Quests)

ORTA

Özellik: "5 kutu kır", "200 altın topla", "1 alet bul" gibi her gün yenilenen 3 görev. Tamamlanınca elmas/altın ödülü verir.

Gerekli Paketler: shared_preferences: ^2.3.0

Etkilenen/Yeni Dosyalar:

- lib/features/quests/domain/models/daily_quest_model.dart (yeni)
- lib/features/quests/application/daily_quest_notifier.dart (yeni)
- lib/features/quests/presentation/widgets/quest_panel.dart (yeni)

Teknik Uygulama:

- Yeni bir DailyQuestModel (id, description, targetValue, currentValue, rewardGold, rewardGems, isCompleted) tanımlanır.
- Yeni bir DailyQuestNotifier (Riverpod StateNotifier) günün tarihine göre 3 rastgele görev üretir ve SharedPreferences'a son üretim tarihini kaydeder (gün değişince sıfırlar).
- GameNotifier'daki digTargetTile(), purchaseUpgrade() gibi metotlara görev ilerlemesini güncelleyen ref.read(dailyQuestProvider.notifier).trackProgress(...) çağrıları eklenir.
- Ana menüye veya HUD'a küçük bir görev ikonu/rozeti eklenir.

#8 Başarım (Achievement) Sistemi

ORTA

Özellik: "İlk elmas kazma bulma", "1000 kutu kırma", "10. rütbeye ulaşma" gibi kalıcı, tek seferlik hedefler. Rozet koleksiyonu şeklinde gösterilir.

Etkilenen/Yeni Dosyalar:

- lib/features/achievements/domain/models/achievement_model.dart (yeni)
- lib/features/achievements/application/achievement_notifier.dart (yeni)
- lib/features/mining/domain/models/player_state_model.dart (düzenleme — yeni sayaçlar)

Teknik Uygulama:

- AchievementModel (id, title, description, iconEmoji, targetValue, isUnlocked) tanımlanır; statik bir liste olarak achievements_catalog.dart içinde tutulur.
- AchievementNotifier, PlayerStateModel'deki lifetimeEarnings, rank, tilesCleared (yeni kümülatif sayaç eklenmeli) gibi alanları dinleyerek eşik aşıldığında rozeti açar.
- PlayerStateModel'e totalTilesBroken, totalToolsFound gibi kümülatif sayaçlar eklenmesi gerekir (şu an bu bilgiler tutulmuyor).
- Kilit açıldığında ekranda kısa bir toast/banner animasyonu gösterilir.

#9 Alet Yükseltme / Geliştirme Sistemi

DÜŞÜK

Özellik: Envanterdeki mevcut aletler (kazma, balta, kürek vb.) altın harcanarak seviye atlatılabilir; sadece yeni alet bulmak yerine elde tutulan alete yatırım yapılabilir.

Etkilenen/Yeni Dosyalar:

- lib/features/mining/domain/models/player_state_model.dart (düzenleme)
- lib/features/mining/domain/models/tool_model.dart (düzenleme)
- lib/features/mining/application/game_notifier.dart (düzenleme)
- lib/features/mining/presentation/widgets/inventory_dialog.dart (düzenleme)

Teknik Uygulama:

- ToolType bazlı bir Map toolLevels alanı PlayerStateModel'e eklenir.
- ToolTypeExtension.tileDamage ve pvpDamage hesaplamaları, seviyeye göre çarpan uygulayacak şekilde güncellenir (ör. temel hasar * (1 + level * 0.15)).
- InventoryDialog'a her alet için 'Yükselt' butonu ve maliyet gösterimi eklenir; GameNotifier'a upgradeTool(ToolType type) metodu eklenir.

#10 Kombo / Kritik Vuruş Sistemi

DÜŞÜK

Özellik: Art arda kesintisiz kazı yapıldığında kombo sayacı artar; belirli eşiklerde kritik vuruş şansı ve ödül çarpanı devreye girer.

Etkilenen/Yeni Dosyalar:

- lib/features/mining/application/game_notifier.dart (düzenleme)
- lib/features/mining/presentation/widgets/hud_bar.dart (düzenleme)

Teknik Uygulama:

- GameState'e comboCount ve lastDigTimestamp alanları eklenir.
- digTargetTile() çağrıldığında, son vuruşla aradaki süre 2 saniyeden azsa comboCount artırılır, aksi halde sıfırlanır.
- comboCount belirli eşikleri (5, 10, 20) geçtiğinde damage ve goldReward üzerine çarpan uygulanır (ör. %10, %25, %50 bonus).
- HUD'da küçük bir 'COMBO x5' göstergesi eklenir (hud_bar.dart genişletilir).

#11 Tehlikeli Özel Bölgeler (Lav / Buz Katmanları)

YÜKSEK

Özellik: Belirli derinliklerde biome'a özgü tehlikeli ama yüksek ödüllü alanlar: lav bölgesinde zamanla can kaybı ama 2x altın, buz bölgesinde yavaşlama ama nadir eşya şansı artışı.

Etkilenen/Yeni Dosyalar:

- lib/features/mining/domain/models/grid_model.dart (düzenleme)
- lib/features/mining/domain/services/grid_generator.dart (düzenleme)
- lib/features/mining/application/game_notifier.dart (düzenleme)
- lib/features/mining/presentation/painters/tile_gridPainter.dart (düzenleme)

Teknik Uygulama:

- GridModel'e biomeHazard enum'ı (none, lava, ice) eklenir; GridGenerator biome adına göre bu değeri set eder (mevcut nextBiome mantığı zaten stage'e göre biome belirliyor).
- GameNotifier'da periyodik bir Timer, oyuncu lav bölgesindeyken hp'yi azaltacak şekilde genişletilir (mevcut _energyTimer deseniyle aynı yapı).
- Buz bölgesinde hareket/kazı maliyeti (enerji tüketimi) artırılır, ama loot tablosundaki nadir ödül olasılıkları yükseltilir.
- tile_gridPainter.dart'a biome'a özgü zemin renk paleti eklenir (AppColors'a yeni sabitler).

Bölüm 3 — Multiplayer ve Sosyal Özellikler

Şu anki multiplayer tamamen yerel simülasyon (mock). Bu bölüm gerçek ağ altyapısına geçişi kapsar.

#12 Gerçek Zamanlı Multiplayer (Firebase / WebSocket)

YÜKSEK

Özellik: Şu an LobbyNotifier ve GameNotifier içindeki 'diğer oyuncular' tamamen yerel rastgele simülasyondur (simulateTeammatesAction()). Bu özellik gerçek cihazlar arası senkron oyunu sağlar.

Gerekli Paketler: `firebase_core: ^3.6.0` · `cloud_firestore: ^5.4.0` · `firebase_auth: ^5.3.0` (anonim giriş için)

Etkilenen/Yeni Dosyalar:

- `lib/core/network/firestore_room_service.dart` (yeni)
- `lib/features/multiplayer/application/lobby_notifier.dart` (yeniden yazım)
- `lib/features/mining/application/game_notifier.dart` (düzenleme — `simulateTeammatesAction` kaldırma)

Teknik Uygulama:

- Backend seçimi: Cloud Firestore (kolay entegrasyon, gerçek zamanlı listener desteği) ya da özel bir WebSocket sunucusu (düşük gecikme, daha fazla kontrol) — orta ölçekli oyunlar için Firestore önerilir.
- RoomModel, Firestore'da 'rooms/{roomId}' dokümanı olarak tutulur; players alt-koleksiyon olarak senkronize edilir.
- LobbyNotifier.createRoom()/joinRoom() metodları mock veri üretimi yerine Firestore yazma/okuma çağrılarına dönüştürülür; oda güncellemeleri StreamProvider ile dinlenir.
- GameNotifier.simulateTeammatesAction() tamamen kaldırılır; yerine her oyuncunun kendi hamlelerini Firestore'a yazdığı, diğer oyuncuların ise stream üzerinden GridModel.otherPlayers listesine yansıtıldığı bir yapı kurulur.
- Çakışma yönetimi (aynı tile'a iki oyuncu aynı anda vurursa) için Firestore transaction kullanılmalı.

#13 Klan / Takım Sistemi

YÜKSEK

Özellik: Oyuncuların kalıcı gruplar (klan) kurup birlikte ilerleyebildiği, klan içi sıralama ve ortak hedeflerin olduğu bir yapı.

Gerekli Paketler: `cloud_firestore` (Bulgu #12 ile paylaşılır)

Etkilenen/Yeni Dosyalar:

- `lib/features/klan/domain/models/klan_model.dart` (yeni)
- `lib/features/klan/application/klan_notifier.dart` (yeni)
- `lib/features/klan/presentation/screens/klan_screen.dart` (yeni)

Teknik Uygulama:

- Bulgu #12'deki gerçek backend altyapısı ön koşuldur.
- Yeni `ClanModel` (`clanId`, `name`, `memberUids`, `totalDepth`, `level`) Firestore'da '`clans/{clanId}`' olarak tutulur.
- `ClanNotifier` ile klan oluşturma/katılma/ayrılma ve klan içi üye listesi yönetimi sağlanır.
- Klan üyelerinin toplam katkısına göre haftalık klan hedefleri (ör. 'Klan olarak toplam 50.000 kutu kırın') ve ortak ödül dağıtımı eklenebilir.

#14 Liderlik Tablosu (Leaderboard)

ORTA

Özellik: En yüksek derinlik, en çok altın, en yüksek rütbe gibi kategorilerde global veya arkadaş sıralaması gösteren bir ekran.

Gerekli Paketler: `cloud_firestore` (Bulgu #12 ile paylaşılır)

Etkilenen/Yeni Dosyalar:

- `lib/features/leaderboard/application/leaderboard_notifier.dart` (yeni)
- `lib/features/leaderboard/presentation/screens/leaderboard_screen.dart` (yeni)

Teknik Uygulama:

- Firestore'da '`leaderboard/{uid}`' dokümanı, oyuncunun `highestDepth/lifetimeEarnings/rank` alanlarını tutar; `PlayerStateModel` her önemli değişiklikte bu dokümana yazar.
- Basit versiyon için Firestore query (`orderBy` + `limit`) ile top 100 çekilir; ölçek büyürse Cloud Functions ile periyodik önceden hesaplanmış sıralama tablosu oluşturulabilir.
- `LeaderboardScreen`, sekmeli yapı (Derinlik / Altın / Rütbe) ile üç farklı sıralama gösterir.
- Kendi sırasını her zaman görünür tutmak için oyuncunun satırı listede sabitlenebilir (`sticky row`).

#15 Oyun İçi Sohbet ve Emoji Tepkileri

DÜŞÜK

Özellik: Oda/lobi içinde hazır emoji tepkileri (begeni, kazma, altın, gulme) veya kısa hazır mesajlarla hızlı iletişim; serbest metin sohbeti spam/moderasyon riski nedeniyle ilk aşamada önerilmez.

Gerekli Paketler: `cloud_firestore` (Bulgu #12 ile paylaşılır)

Etkilenen/Yeni Dosyalar:

- `lib/features/multiplayer/presentation/widgets/reaction_bar.dart` (yeni)
- `lib/features/multiplayer/application/reaction_notifier.dart` (yeni)

Teknik Uygulama:

- Firestore'da `'rooms/{roomId}/reactions'` alt-koleksiyonuna zaman damgalı, oyuncu id'li kısa kayıtlar yazılır (ör. `{uid, emoji, timestamp}`).
- `LobbyScreen` ve `MiningScreen`'e ekranın altında sabit bir emoji şeridi eklenir; basılan emoji ekranda diğer oyuncuların üzerinde kısa süreli bir balon olarak belirir.
- Hazır mesaj seçenekleri ("Yardım!", "Buraya gel!", "İyi iş!") aynı mekanizmayla gönderilebilir — serbest metin girişi olmadığı için ek moderasyon yükü getirmez.

Bölüm 4 — Monetizasyon ve Yaşam Döngüsü

Oyunun sürdürülebilirliği ve uzun vadeli oyuncu bağlılığı için gelir modelleri.

#16 Reklam ile Enerji Yenileme

ORTA

Özellik: Enerji bittiğinde oyuncuya ödüllü reklam (rewarded ad) izleyerek anında enerji dolumu veya geçici 2x kazı hızı bonusu sunulur.

Gerekli Paketler: `google_mobile_ads: ^5.2.0`

Etkilenen/Yeni Dosyalar:

- `lib/core/ads/ad_service.dart` (yeni)
- `lib/features/mining/application/game_notifier.dart` (düzenleme)
- `lib/features/mining/presentation/widgets/bottom_progress_bar.dart` (düzenleme — buton)

Teknik Uygulama:

- `google_mobile_ads` paketi entegre edilir; AdMob hesabı ve rewarded ad unit ID'leri alınır.
- `GameNotifier`'a `watchAdForEnergy()` metodu eklenir; reklam tamamlanma callback'inde `regenerateEnergy()` zaten var olan metod çağrılır.
- Enerji 0 olduğunda gösterilen mevcut uyarı mesajının yanına (`digTargetTile()` içindeki 'Enerji tükendi!' bloğu) bir 'REKLAM İZLE' butonu eklenir.
- Reklam yüklenemezse (bağlantı yok vb.) zarif bir fallback (buton devre dışı + bekleme mesajı) gösterilmelidir.

#17 Kozmetik Market (Karakter Özelleştirme)

ORTA

Özellik: Kask rengi, tişört/pantolon rengi, özel karakter derisi gibi salt görsel, oyun dengesini bozmayan satın alınabilir öğeler.

Gerekli Paketler: `in_app_purchase: ^3.2.0` (gerçek para ile satış isteniyorsa)

Etkilenen/Yeni Dosyalar:

- `lib/features/cosmetics/domain/models/cosmetic_item_model.dart` (yeni)
- `lib/features/cosmetics/presentation/screens/cosmetic_shop_screen.dart` (yeni)
- `lib/features/mining/presentation/painters/tile_gridPainter.dart` (düzenleme)
- `lib/features/mining/domain/models/player_state_model.dart` (düzenleme)

Teknik Uygulama:

- `PlayerStateModel`'e `equippedSkinId`: String alanı eklenir.
- `tile_gridPainter.dart`'taki `_drawMinerCharacter()` metodu, sabit `shirtColor/pantsColor` yerine `equippedSkinId`'ye göre bir `CosmeticCatalog`'dan renk/görsel okuyacak şekilde genişletilir.
- Uygulama içi satın alma için `in_app_purchase` paketi ile gerçek para (App Store/Play Store) veya oyun içi elmas karşılığı satın alma desteklenir.
- `CosmeticShopScreen`, mevcut `ShopPanel` deseniyle tutarlı bir arayüzle eklenir.

#18 Sezonluk İçerik Geçişleri (Battle Pass Benzeri)

YÜKSEK

Özellik: Belirli aralıklarla (ör. 4 haftada bir) yeni biome/tema, sınırlı süreli kozmetikler ve kademeli ödül çizelgesi (ücretsiz + premium hat) sunan bir sezon sistemi.

Gerekli Paketler: `cloud_firestore` · `in_app_purchase` (Bulgu #12, #17 ile paylaşılır)

Etkilenen/Yeni Dosyalar:

- `lib/features/season/domain/models/season_model.dart` (yeni)
- `lib/features/season/application/season_notifier.dart` (yeni)
- `lib/features/season/presentation/screens/season_pass_screen.dart` (yeni)

Teknik Uygulama:

- `SeasonModel` (`seasonId`, `startDate`, `endDate`, `tiers: List`) tanımlanır; sunucu tarafında (Firestore) tutulup istemci tarafından okunur.
- `PlayerStateModel`'e `seasonXp: int` eklenir; `digTargetTile()`, `purchaseUpgrade()` gibi mevcut eylemler küçük miktarlarda `seasonXp` de kazandıracak şekilde genişletilir.
- `SeasonPassScreen`, biriken XP'ye göre açılan ücretsiz/premium ödül basamaklarını gösterir.
- Premium hat için `in_app_purchase` entegrasyonu (Bulgu #17 ile ortak altyapı) kullanılabilir.
- Bu özellik, Bulgu #12'deki gerçek backend ve Bulgu #7'deki görev sistemi altyapısının üzerine inşa edilmelidir — bağımsız yapılırsa veri tutarlılığı zorlaşır.

Öneri: Uygulama sırasında önce Bölüm 1 (Öncelikli) tamamlanmalı, ardından Bulgu #12 (gerçek zamanlı multiplayer altyapısı) kurulmalıdır — çünkü Bölüm 3 ve Bölüm 4'teki birçok özellik (klan, liderlik tablosu, sezon sistemi) bu backend'e bağımlıdır. Oynanış derinliği (Bölüm 2) özellikleri ise backend'den bağımsız olarak paralel geliştirilebilir.